

Python OOP Practice Questions

Classes & Objects | Methods | Encapsulation | Inheritance

This workbook contains practice questions covering only the four topics you have studied so far:
OOP Basics, Methods, Encapsulation, and Inheritance.

Difficulty increases gradually within each section, from very simple to challenging. Some questions include an Expected Output box so you can check your work. Others do not, so that you can practice predicting or designing the output yourself.

Section	Topic	Questions
1	OOP Introduction / Classes & Objects	1 - 12
2	Methods	13 - 24
3	Encapsulation	25 - 34
4	Inheritance	35 - 46
5	Mixed OOP Questions	47 - 54
6	Final Challenge	55 - 59

How to use this workbook: solve each question yourself first. Do not look for the code answer — this PDF only contains questions. Use the Expected Output boxes (where given) only to check your final result after you have written and run your own code.

Section 1: OOP Introduction & Classes / Objects

These questions focus on creating classes, creating objects, setting attributes, and accessing object data. Start from Question 1 and move in order.

Question 1

Create an empty class called **Car** (use pass inside it). Create one object of this class. Print the **type** of the object using `type(object_name)`.

Expected Output:

```
<class '__main__.Car'>
```

Question 2

Create a class called **Dog** with two attributes, name and breed, set inside `__init__`. Create one object with name = "Buddy" and breed = "Labrador". Print both attributes, one per line.

Expected Output:

```
Buddy
Labrador
```

Question 3

Create a class called **Book** with attributes `title`, `author`, and `price`. Create an object for the book "Python Basics" by "John Smith" priced at 350. Print the details in this exact format:

Title: <title>

Author: <author>

Price: <price>

Expected Output:

```
Title: Python Basics
Author: John Smith
Price: 350
```

Question 4

Create a class called **Rectangle** with attributes `length` and `width`. Create **two different objects** with values of your own choice, and print both objects' length and width.

Question 5

Create a class called **Car** with attributes `brand`, `model`, and `year`. Create **three** car objects with values of your own choice and print the details of each one.

Question 6

Create a class called **Circle** with a single attribute `radius`. Create an object with `radius = 5`. After creating the object, change the radius to 10 using dot notation (`object.radius = 10`). Print the updated radius.

Expected Output:

```
10
```

Question 7

Create a class called **Employee** with attributes `name`, `salary`, and `department`. Create one object. **After** creating the object, add a brand-new attribute called `bonus` to it from outside the class. Print all four attributes (`name`, `salary`, `department`, `bonus`).

Question 8

Create a class called **Point** with attributes `x` and `y`. Create two objects: `point1` with `x=2`, `y=3` and `point2` with `x=5`, `y=7`. Print a message for each point in this exact format:

Point1: (2, 3)

Point2: (5, 7)

Expected Output:

```
Point1: (2, 3)
```

```
Point2: (5, 7)
```

Question 9

Create a class called **BankAccount** with attributes `account_holder` and `balance`. Create one object with `account_holder = "Sita"` and `balance = 5000`. Print the details in this exact format:

Account Holder: Sita

Balance: 5000

Then create a **second** object with values of your own choice and print its details too (this second part has no fixed output — use your own values).

Expected Output:

```
Account Holder: Sita
```

```
Balance: 5000
```

Question 10

Create a class called **Movie** with attributes `title`, `genre`, and `rating`. Create a **list** containing three Movie objects with values of your own choice. Loop through the list and print only each movie's `title`.

Question 11

Create a class called **Product** with attributes `name`, `price`, and `quantity`. Create an object with `name = "Pen"`, `price = 10`, `quantity = 5`. Without writing any method, print the total cost (price multiplied by quantity) directly inside a print statement, in this format:

Total Cost: 50

Expected Output:

```
Total Cost: 50
```

Question 12

Create a class called **Library** with a single attribute `books`, which should start as an **empty list** when an object is created. After creating the object, use `.append()` to add three book names of your choice to the list. Print the full list at the end.

Section 2: Methods

These questions focus on writing methods inside a class, calling them, using parameters, and using an object's own data inside its methods.

Question 13

Create a class called **Greeter** with a method `say_hello()` that prints "Hello!" when called. Create an object and call the method.

Expected Output:

```
Hello!
```

Question 14

Create a class called **Student** with attributes `name` and `age` set inside `__init__`. Add a method `display()` that prints the name and age, each on its own line. Create an object with `name = "Anita"`, `age = 22` and call `display()`.

Expected Output:

```
Anita
22
```

Question 15

Create a class called **Calculator** with a method `add(a, b)` that **returns** the sum of two numbers. Create an object, call `add(4, 5)`, store the result in a variable, and print it.

Expected Output:

```
9
```

Question 16

Create a class called **Circle** with attribute `radius` set inside `__init__`. Add a method `area()` that calculates and returns the area using the formula $3.14 * \text{radius} * \text{radius}$. Create an object with `radius = 4` and print the result of calling `area()`.

Question 17

Create a class called **Rectangle** with attributes `length` and `width`. Add two methods: `area()` and `perimeter()`. Create an object with `length = 6`, `width = 3`, and print the results of calling both methods.

Question 18

Create a class called **Employee** with attributes `name` and `salary`. Add a method `give_raise(amount)` that increases the salary by the given amount and then prints the new salary. Create an object with `salary = 30000` and call `give_raise(5000)`.

Expected Output:

```
35000
```

Question 19

Create a class called **Counter** with an attribute `count` that starts at 0. Add a method `increment()` that increases `count` by 1 every time it is called. Call `increment()` three times, then print the final `count`.

Expected Output:

```
3
```

Question 20

Create a class called **Temperature** with attribute `celsius`. Add a method `to_fahrenheit()` that returns the Fahrenheit value using the formula $(celsius * 9/5) + 32$. Create an object with `celsius = 25` and print the result.

Question 21

Create a class called **ShoppingCart** with attribute `items` (an empty list, set inside `__init__`). Add a method `add_item(item)` that appends an item to the list, and a method `show_items()` that prints every item, one per line. Add "Book", "Pen", and "Bag", then call `show_items()`.

Expected Output:

```
Book
Pen
Bag
```

Question 22

Create a class called **BankAccount** with attribute `balance` starting at 0. Add a method `deposit(amount)` that adds to the balance, and a method `withdraw(amount)` that only allows the withdrawal if there is enough balance; otherwise it should print "Insufficient balance". Test it by depositing 1000, withdrawing 300, and then trying to withdraw 2000.

Question 23

Create a class called **Student** with attribute `name` and a list `marks`. Add a method `average()` that calculates and returns the average of the marks list. Create an object with `marks = [80, 90, 70]` and print the average.

Expected Output:

```
80.0
```

Question 24

Create a class called **Library** with attribute `books` (an empty list). Add three methods: `add_book(title)`, `remove_book(title)`, and `total_books()` (which returns the current count of books). Add 4 books, remove 1 of them, then print the total count using `total_books()`.

Section 3: Encapsulation

These questions focus on protecting data using private and protected-style attributes, and on reading or updating that data safely through methods.

Question 25

Create a class called **Person** with a **private** attribute `__name`, set inside `__init__`. Add a method `get_name()` that returns the value of `__name`. Create an object with `name = "Kiran"` and print the result of calling `get_name()`.

Expected Output:

```
Kiran
```

Question 26

Using the same idea as the previous question, create a class called **Person** with a private attribute `__age`. After creating an object, try to print `object.__age` directly from outside the class (not through a method). Run your code and read the error message carefully. Write one line in a comment explaining what the error tells you about private attributes.

Question 27

Create a class called **BankAccount** with a private attribute `__balance` set inside `__init__`. Add a method `get_balance()` that returns the balance, and a method `deposit(amount)` that increases it. Create an object with `balance = 1000`, `deposit 500`, then print the balance using `get_balance()`.

Expected Output:

```
1500
```

Question 28

Create a class called **BankAccount** with a private attribute `__balance`. Add a method `withdraw(amount)` that only allows withdrawal if `amount <= balance`; otherwise it should print "Insufficient funds". Also add `get_balance()`. Test with `balance = 2000`: `withdraw 500`, then try to `withdraw 3000`.

Question 29

Create a class called **Employee** with a **protected** attribute `_salary` (single underscore) set to 40000. Add a method `show_salary()` that prints "Salary: 40000". Create an object, call `show_salary()`, and then also print `object._salary` directly from outside the class, to see that protected attributes (unlike private ones) can still be accessed.

Expected Output:

```
Salary: 40000
40000
```

Question 30

Create a class called **Student** with private attributes `__name` and `__marks`. Add a method `update_marks(new_marks)` that changes the marks, and a method `get_details()` that prints the name and marks, each on its own line. Create an object with `name = "Meena"`, `marks = 75`. Update the marks to 85, then call `get_details()`.

Expected Output:

```
Meena
85
```

Question 31

Create a class called **BankAccount** with a private attribute `__balance`. Add a method `deposit(amount)` that only accepts **positive** amounts. If the amount is zero or negative, it should print "Invalid deposit amount" instead of changing the balance. Test it with `deposit(500)` and `deposit(-100)`.

Question 32

Create a class called **Product** with private attributes `__name` and `__price`. Add a method `get_price()` and a method `set_price(new_price)`. The `set_price` method should refuse to set a price below 0 (print "Price cannot be negative" in that case). Create a product with `price = 100`, try to set it to -50 (should fail), then set it to 150, and finally print the price using `get_price()`.

Question 33

Create a class called **Password** with a private attribute `__password`. Add a method `check_password(input_password)` that returns `True` if the input matches the stored password, and `False` otherwise. Create an object with `password = "abc123"`. Test it by calling `check_password("abc123")` and `check_password("wrong")`, and print both results. Try to predict the two outputs before you run the code.

Question 34

Create a class called **Student** with private attributes `__name` and `__age`. Add methods `get_name()`, `get_age()`, and `set_age(new_age)`. The `set_age` method should not accept a negative age — in that case it should print "Invalid age" and leave the age unchanged. Create an object with `name = "Ravi"`, `age = 18`. Call `set_age(-5)`, then call `set_age(19)`, then print the final age using `get_age()`.

Expected Output:

```
Invalid age
19
```

Section 4: Inheritance

These questions focus on parent and child classes: inheriting attributes and methods, using `super()`, and overriding methods in a child class.

Question 35

Create a parent class called **Animal** with a method `speak()` that prints "Animals can make sounds". Create a child class called **Dog** that inherits from `Animal` (leave it empty with `pass`). Create a `Dog` object and call `speak()` on it.

Expected Output:

```
Animals can make sounds
```

Question 36

Create a parent class called **Vehicle** with attribute `brand` (set inside `__init__`) and a method `show_brand()` that prints the brand. Create a child class called **Car** that inherits from `Vehicle`. Create a `Car` object with `brand = "Toyota"` and call `show_brand()`.

Expected Output:

```
Toyota
```

Question 37

Create a parent class called **Person** with attributes `name` and `age`, and a method `display()` that prints both, each on its own line. Create a child class called **Student** that inherits from `Person` and adds a new attribute `grade`. Give `Student` its own `__init__` that accepts `name`, `age`, and `grade`, and calls the parent's `__init__` using `super()`. Add a method `show_grade()` in `Student`. Create an object and call both `display()` and `show_grade()`.

Question 38

Create a parent class called **Shape** with a method `info()` that prints "This is a shape". Create a child class called **Square** that inherits from `Shape` and adds its own new method `describe()` that prints "This is a square". Create a `Square` object and call **both** `info()` and `describe()` on it.

Expected Output:

```
This is a shape
This is a square
```

Question 39

Create a parent class called **Animal** with a method `sound()` that prints "Some generic animal sound". Create a child class called **Cat** that inherits from `Animal` but **overrides** `sound()` to print "Meow" instead. Create one object of `Animal` and one object of `Cat`, and call `sound()` on each.

Expected Output:

```
Some generic animal sound
Meow
```

Question 40

Create a parent class called **Employee** with attributes `name` and `salary`, and a method `show_salary()`. Create a child class called **Manager** that inherits from **Employee** and adds an extra attribute `department`. Use `super().__init__()` to set `name` and `salary`. Create a manager with `name = "Sunil"`, `salary = 60000`, `department = "Sales"`. Call `show_salary()` and then print the `department` separately.

Question 41

Create a parent class called **Animal** with a method `speak()` that prints "Animal speaks". Create a child class called **Dog** that **overrides** `speak()`, but inside the new version, first calls the parent's original method using `super().speak()`, and then prints "Dog barks" on the next line. Create a **Dog** object and call `speak()`.

Expected Output:

```
Animal speaks
Dog barks
```

Question 42

Create a parent class called **BankAccount** with a protected attribute `_balance`, and methods `deposit(amount)` and `show_balance()`. Create a child class called **SavingsAccount** that inherits from **BankAccount** and adds a new method `add_interest(rate)` that increases the balance by $(\text{balance} * \text{rate} / 100)$. Create an object with `balance = 1000`, add 10% interest, then call `show_balance()`.

Question 43

Create a parent class called **Person** with attribute `name`. Create a child class called **Teacher** that inherits from **Person** and adds an extra attribute `subject`. Add a method `teach()` inside **Teacher** that prints in this exact format:

<name> teaches <subject>

Create a **Teacher** object with `name = "Aman"` and `subject = "Math"`, then call `teach()`.

Expected Output:

```
Aman teaches Math
```

Question 44

Create a parent class called **Shape** with a method `area()` that prints "Area not defined". Create **two** child classes, **Circle** and **Square**, each **overriding** `area()` with its own message — "Circle area logic" for **Circle**, and "Square area logic" for **Square**. Create one object of each class and call `area()` on both.

Question 45

Create a parent class called **Vehicle** with attributes `brand` and `speed`, and a method `show_info()` that prints both. Create a child class called **Bike** that inherits from `Vehicle`, uses `super().__init__()`, and adds its own attribute `has_gear` (True or False). Override `show_info()` inside `Bike` so that it first calls the parent's version using `super().show_info()`, and then also prints whether the bike has gears. Create an object with `brand = "Yamaha"`, `speed = 120`, `has_gear = True`.

Question 46

Create a parent class called **Employee** with attributes `name` and `salary`, a method `show_salary()` that prints in the format "Salary: <salary>", and a method `give_bonus(amount)` that increases the salary. Create a child class called **Manager** that inherits everything from `Employee` without overriding anything, and simply adds one new attribute, `team_size`. Create a `Manager` object with `name = "Divya"`, `salary = 50000`, `team_size = 5`. Call the inherited `give_bonus(5000)`, then `show_salary()`. Finally, print the team size using: `print("Team Size:", team_size)`.

Expected Output:

```
Salary: 55000
Team Size: 5
```

Section 5: Mixed OOP Questions

These questions combine everything you have learned so far — classes, objects, methods, encapsulation, and inheritance — in a single problem. Read each question carefully before you start coding.

Question 47

Create a class called **Person** with private attributes `__name` and `__age`, set through `__init__`. Add methods `get_name()`, `get_age()`, and `display()` (which prints both using the getter methods). Create a child class called **Employee** that inherits from **Person**, adds a new private attribute `__salary`, and a method `show_salary()`. Use `super().__init__()` to set name and age. Create an **Employee** object with name = "Rohit", age = 30, salary = 45000. Call `display()` and `show_salary()`.

Question 48

Create a class called **Shape** with a method `area()` that prints "Shape area not defined". Create a child class called **Rectangle** with attributes `length` and `width`, and **override** `area()` so it calculates and prints the actual area in this format: "Area: <value>". Create an object with length = 5, width = 4, and call `area()`.

Expected Output:

```
Area: 20
```

Question 49

Create a parent class called **Account** with a protected attribute `_balance`, and methods `deposit(amount)` and `get_balance()`. Create a child class called **CurrentAccount** that adds an `overdraft_limit` attribute and **overrides** a method `withdraw(amount)` so that a withdrawal is allowed even if the balance goes negative, as long as it does not go below `-overdraft_limit`. Create an object with balance = 1000, `overdraft_limit` = 500. Try withdrawing 1300 (this should succeed, balance becomes -300), then try withdrawing another 500 (this should fail because it would exceed the overdraft limit — print a message such as "Overdraft limit exceeded").

Question 50

Create a class called **Book** with attributes `title` and `author`. Create a class called **Library** with an empty list `books`, a method `add_book(book)` that adds a **Book** object to the list, and a method `show_books()` that prints the title and author of every book, one per line. Add 3 books of your choice and call `show_books()`.

Question 51

Create a parent class called **Staff** with a private attribute `__name` and a protected attribute `_salary`. Add a method `get_name()` and a method `show_salary()`. Create a child class called **Intern** that inherits from **Staff**, adds an attribute `stipend_days`, and a new method `total_pay()` that returns `_salary` (accessed directly, since it is protected, not private). Create an **Intern** object with

name = "Zoya", salary = 200, stipend_days = 30. Print the result of `get_name()`, and then print the result of `total_pay() * stipend_days`.

Expected Output:

```
Zoya
6000
```

Question 52

Create a class called **Animal** with attributes `name` and `sound`, and a method `make_sound()` that prints in this format: "<name> says <sound>". Create two child classes, **Dog** and **Cat**, each using `super().__init__()` to pass a fixed sound value — "Woof" for Dog, "Meow" for Cat — while still accepting `name` as a parameter. Create a Dog named "Rocky" and a Cat named "Tom", then call `make_sound()` on both.

Expected Output:

```
Rocky says Woof
Tom says Meow
```

Question 53

Create a class called **Employee** with a private attribute `__salary` and a method `get_salary()` that returns it. Create a child class called **Manager** that **overrides** `get_salary()` so that it returns the parent's salary **plus** a bonus of 10000 (use `super().get_salary()` inside your override). Create a Manager object with `salary = 50000` and print the result of calling `get_salary()`.

Expected Output:

```
60000
```

Question 54

Build a small "Zoo" system with three classes:

- **Animal** — attributes `name`, `species`, and a method `describe()` that prints "<name> is a <species>".
- **Zoo** — a private list `__animal_list`, a method `add_animal(animal)` that adds an Animal object to the list, and a method `show_all()` that calls `describe()` on every animal in the list. Create three different Animal objects, add them to a Zoo object, and call `show_all()`.

Section 6: Final Challenge

These are the hardest questions in this set. They only use the four topics you already know — there is nothing new to learn here, just careful thinking and careful coding. Take your time.

Question 55

Create a parent class called **Vehicle** with a private attribute `__fuel_level` that starts at 100, a method `drive(distance)` that reduces `__fuel_level` by `distance // 10`, and a method `get_fuel_level()`. Create a child class called **ElectricVehicle** that **overrides** `drive(distance)` so that, instead of using fuel, it reduces a **new** private attribute `__battery_level` (also starting at 100) by `distance // 5`. Add a method `get_battery_level()`. Create an `ElectricVehicle` object, call `drive(50)`, then print the battery level.

Expected Output:

```
90
```

Question 56

Create a class called **Person** with private attributes `__name` and `__age`. Create a child class called **Student** that adds a private list `__marks`, a method `add_marks(mark)` that appends to the list, and a method `average()` that returns the average of the marks. Now create a further child class called **TopperStudent** that inherits from `Student` and **overrides** `average()` so that it adds 5 bonus points to the final result (call `super().average()` first, then add 5). Create a `TopperStudent` object, add the marks 80, 90, and 70, and print the result of calling `average()`. Work out what the normal `Student` average would have been, and compare it with the boosted result you get.

Question 57

Design a simple bank system using two classes:

- **Account** (parent) — a private attribute `__balance`, and methods `deposit(amount)`, `withdraw(amount)` (which must check there is enough balance before withdrawing), and `get_balance()`.
- **SavingsAccount** (child) — adds an attribute `interest_rate` and a method `apply_interest()` that increases the balance by `balance * interest_rate / 100`.

Create a `SavingsAccount` with `balance = 2000`, `interest_rate = 5`. Deposit 500, apply interest, then withdraw 100. Print the final balance using `get_balance()`.

Question 58

Create a parent class called **Shape** with an attribute `name` and a method `area()` that prints "Area method not implemented for <name>". Create two child classes:

- **Circle** — attribute `radius`, overrides `area()` to print "Area of Circle: <value>" using `3.14 * radius * radius`.
- **Rectangle** — attributes `length` and `width`, overrides `area()` to print "Area of Rectangle: <value>" using `length * width`.

Create one Circle with radius = 3, and one Rectangle with length = 4, width = 5. Call `area()` on both, Circle first.

Expected Output:

```
Area of Circle: 28.26
Area of Rectangle: 20
```

Question 59

Build a small library-borrowing system using two classes:

- **Book** — attributes `title` and `is_borrowed` (defaults to `False`).
 - **Member** — a private attribute `__name`, a private list `__borrowed_books`, a method `borrow_book(book)` that only allows borrowing if `book.is_borrowed` is `False` (then it sets `book.is_borrowed = True` and adds the title to `__borrowed_books`; otherwise it prints "Book already borrowed"), and a method `show_borrowed_books()` that prints all borrowed titles.
- Create two Book objects and one Member object. Borrow both books, then try borrowing the first book again (it should print the "already borrowed" message), and finally call `show_borrowed_books()`.

End of Workbook — Good luck with your practice!